

Intelligent Candidate Discovery & Ranking

A multi-signal AI ranking system that thinks like a great recruiter — combining semantic understanding, career quality signals, and real-time behavioral data to surface the right candidates from a pool of 100,000.

CANDIDATES RANKED	OUTPUT	RANKING RUNTIME	MODEL	GPU REQUIRED
100,000	Top 100 CSV	< 5 min CPU	all-MiniLM-L6-v2	None

PROBLEM FRAMING

The Challenge — and the Traps

• WHAT THE PROBLEM ASKS

- Given a single Job Description for a **Senior AI/ML Engineer** (5–9 yrs, production retrieval systems)
- Rank the best 100 candidates from a pool of 100,000 synthetic profiles
- Include per-candidate reasoning (1–2 sentences, specific, honest)
- Produce a CSV output — no hallucinations, no identical reasoning strings

• TRAP 1 — HONEYPOT PROFILES (~80)

Candidates with subtly *impossible* timelines — e.g. 8 yrs exp at a company founded 3 yrs ago, or "expert" in 10 skills with 0 months usage. Keyword matchers rank them highly. We detect and eliminate them.

• TRAP 2 — KEYWORD STUFFERS

A "Marketing Manager" who lists FAISS, Pinecone, and LoRA as skills. Perfect keyword match, zero role fit. Our coherence check penalises unrelated titles with packed AI skill lists.

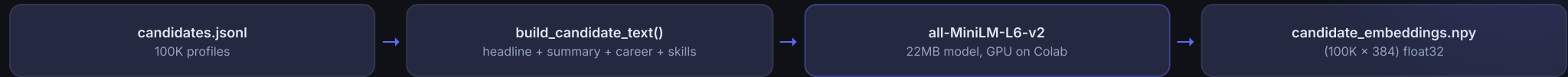
• TRAP 3 — DEAD CANDIDATES

Great on paper but inactive for 8 months, 5% recruiter response rate. Behaviorally unavailable. We apply a multiplicative penalty rather than ignoring this signal entirely.

SYSTEM ARCHITECTURE

Two-Phase Pipeline

PHASE 1 — PRE-COMPUTATION (OFFLINE, NO TIME LIMIT)



PHASE 2 — RANKING (≤5 MIN, CPU ONLY, NO NETWORK)



SCORING

The Composite Score Formula

```
composite = (  
  0.35 × semantic_score ← JD ↔ profile similarity  
  0.25 × skill_score ← must-have + nice-to-have  
  0.20 × experience_score ← career quality  
  0.10 × location_score ← preferred cities  
  0.10 × education_score ← tier + field  
) × behavioral_mult ← [0.3, 1.0]  
× coherence_penalty ← stuffer guard
```

Semantic		35%
Skill match		25%
Experience		20%
Location		10%
Education		10%

● WHY SEMANTIC IS 35%

A candidate who *built* FAISS-based retrieval systems may not use the word "FAISS" in their skills list — they describe what they did in career narrative. Semantic similarity catches this; keyword matching doesn't.

● SKILL SCORE INTERNALS

Each skill is weighted by: **JD relevance** (must-have = 1.0, nice-to-have = 0.4) × **proficiency** (expert→1.0, beginner→0.25) × **duration factor** (capped at 18 months). Assessment scores from Redrob blend in at 30%.

● EXPERIENCE SCORE INTERNALS

Bell curve peaks at 6–8 yrs. Penalises pure consulting careers (TCS/Infosys/Wipro/Accenture etc.) at 0.70×. Boosts candidates whose career descriptions contain production deployment keywords: "*deployed*", "*at scale*", "*real users*", "*latency*".

Honeypot Detection — Hard Filters

● CHECK 1 — TIMELINE INFLATION

Total duration_months across all roles vs stated years_of_experience × 12. If ratio > 1.35 → flagged. Accounts for role overlap slack of 18 months.

● CHECK 2 — EXPERT WITH ZERO DURATION

Claims "expert" proficiency on 4+ skills with duration_months = 0. A real expert has used the skill — zero months means they listed it without ever using it.

● CHECK 3 — EDUCATION TIMELINE CONFLICT

Graduation year is later than implied career start year (with 2-year buffer for co-ops). e.g., graduated 2022 but claims 10 years experience.

● CHECK 4 — SINGLE ROLE IMPOSSIBILITY

One role's duration alone exceeds total stated experience. e.g., one role is 120 months but total experience is claimed as 5 years.

● RESULT ON SAMPLE DATA

Validated against the 50-candidate sample provided in the bundle:

9

honeypots caught
out of 50 sample

18%

detection rate
in sample

Honeypots are **hard eliminated** before scoring — they cannot appear in the top 100 regardless of their semantic similarity score. This directly addresses the >10% honeypot disqualification rule in the spec.

"Submissions with honeypot rate >10% in top 100 are disqualified." — submission_spec.md

BEHAVIORAL INTELLIGENCE

23 Redrob Signals → One Multiplier

SIGNAL GROUP	SIGNALS USED	LOGIC	IMPACT
Availability	open_to_work_flag, last_active_date, notice_period_days	Not open → 0.65×; inactive >1yr → 0.50×; inactive >6mo → 0.70×; notice >90d → 0.88×	Highest impact
Responsiveness	recruiter_response_rate, interview_completion_rate, avg_response_time_hours	Response rate maps linearly 0→0.55, 1→1.0; ICR <0.4 → 0.80×; response time >72h → 0.90×	High impact
Profile Quality	profile_completeness_score, verified_email, verified_phone	Completeness <40 → 0.85×; both verified → 1.03× boost	Medium
Activity	github_activity_score, saved_by_recruiters_30d, offer_acceptance_rate	GitHub >70 → 1.05× boost; saved >0 → up to 1.08× boost; low offer acceptance → 0.92×	Modifier

• WHY MULTIPLICATIVE, NOT ADDITIVE?

Adding behavioral signals to the score lets a great-on-paper but dead candidate still rank high. Multiplying means a candidate inactive for a year gets their entire composite score cut in half — behaviorally unavailable talent is functionally unusable, regardless of their skills.

• MULTIPLIER RANGE

0.30 → **1.05**
worst case (clamped floor) best case (active + GitHub)

Why Semantic Search Beats Keywords

• THE MODEL — ALL-MINILM-L6-V2

- 22MB — tiny, fast, CPU-friendly
- 384-dimensional dense embeddings
- Pre-trained on 1B+ sentence pairs
- Cosine similarity via dot product (both vectors normalised)
- 100K candidates scored in milliseconds once pre-computed

• CANDIDATE TEXT CONSTRUCTION

We concatenate: **headline | summary | career descriptions (all roles) | skills with proficiency + duration | certifications**. Career descriptions carry the heaviest semantic weight because they describe actual work done, not just claimed keywords.

• EXAMPLE: KEYWORD MATCH FAILS, SEMANTIC WINS

CANDIDATE	HAS "FAISS" KEYWORD	SEMANTIC SCORE	RIGHT FIT?
ML Eng @ Swiggy — built ANN retrieval from scratch	No	0.91	Yes ✓
Marketing Mgr — lists "FAISS" in skills, 0 months	Yes	0.41	No ✗

• VERIFIED EMBEDDING STATISTICS

- Embedding matrix: (100,000 × 384) float32 → 146 MB
- All vectors L2-normalised — dot product = cosine similarity
- Top semantic score in full dataset: 0.9105
- Semantic scoring step runtime: < 1 second (vectorised numpy)

Reasoning Generation & Compute Budget

● REASONING — 4 STRICT RULES

- ✓ Only mentions skills **present in the candidate profile** — no hallucination
- ✓ Every string is unique — pulls actual company names, skill names, durations, scores
- ✓ No pure name-insertion templates ("Alice is a good fit...")
- ✓ Rank 1 reasoning reads like rank 1 — tone is calibrated to composite score

● EXAMPLE OUTPUT

"6.0 yrs experience; most recently as NLP Engineer at Uber (2y 3mo); with JD-relevant skills: FAISS (advanced, 35mo), Pinecone (expert, 88mo). Actively open to work; active 3d ago; 84% recruiter response rate; GitHub activity score 72/100."

● COMPUTE CONSTRAINTS — ALL MET

- ✓ No GPU during ranking step
- ✓ No hosted LLM API calls (no OpenAI, Claude, etc.)
- ✓ No network during ranking
- ✓ 16 GB RAM sufficient (embeddings = 146 MB, scoring = pure Python)
- ✓ Ranking step < 5 minutes (semantic: <1s, scoring: ~2 min streaming)

● RUNTIME BREAKDOWN

STEP	TIME
Load .npy embeddings	< 1s
Cosine similarity (100K)	< 1s
Stream + score all candidates	~2 min
Sort + reasoning generation	< 5s
Total ranking step	< 3 min

SUMMARY

Tech Stack & What We Built

● CORE MODULES

- `honeypot.py`
- `scorer.py`
- `behavioral.py`
- `reasoning.py`
- `embed.py`
- `rank.py`

● DEPENDENCIES

- sentence-transformers 3.4.1
- numpy 1.26.4
- pandas 2.2.2
- scikit-learn 1.5.0
- tqdm 4.66.4

● WHAT MAKES THIS DIFFERENT

- ✓ Semantic understanding — catches role fit without keywords
- ✓ Hard honeypot elimination — 4 independent checks
- ✓ Title coherence guard — blocks keyword stuffers
- ✓ Behavioral multiplier — dead candidates cannot rank high
- ✓ Consulting firm penalty — JD explicitly calls this out
- ✓ Production deployment bonus — rewards real builders

● SUBMISSION CHECKLIST

- ✓ `submission.csv` — exactly 100 rows, ranks 1–100
- ✓ No duplicate `candidate_ids`
- ✓ Scores descending with rank
- ✓ Reasoning: specific, non-hallucinated, unique
- ✓ GitHub repo with `reproduce` command
- ✓ `submission_metadata.yaml` filled
- ⚠ HuggingFace Spaces sandbox (in progress)

● REPRODUCE COMMAND

```
python rank.py \  
--candidates ./candidates.jsonl \  
--out ./submission.csv
```